CS230 Fall 2025 Group 5 Project Report

Fault-Tolerance in a Credit Card Network Through Distributed Systems

Aaditya Borse, Giulia Ghidoli, Jan Groeneveld

{borsea, ghidolig, jgroenev}@uci.edu

1 Introduction

In modern financial ecosystems, credit card networks facilitate billions of secure, real-time transactions everyday. As these networks increasingly grow in size and complexity, they rely on fault-tolerant distributed systems to ensure transactions. This includes complete authorization and fraud detection. The shift toward distributed systems provides benefits for performance and scalability, but introduces added complexity. In response, distributed systems must address challenges of network partitions, crashed processes, and data inconsistencies.

Because credit card networks must also prioritize security of its transactions, fault tolerance is essential. Designing credit card networks to continue operation in the face of component failures involves ensuring liveness, safety, and consensus between non-faulty components. Without these guarantees, a credit card network could not be trusted by its customers. However, according to the FLP theorem, consensus is impossible for asynchronous systems in the presence of even one failure. Thus distributed, asynchronous credit card networks must provide robust fault tolerance and consensus mechanisms to withstand their demands.

Credit card networks also face significant challenges on transaction ingest because of the high volume of e-payments occurring daily. Each transaction must be received, validated, and routed within milliseconds. At the same time, each transaction must be captured exactly once in the presence of retries, network partitions, or partial system failures. This way, users are protected from double charges or conflicts. Every node within the transaction pipeline must provide for some fault tolerance within a credit card network. As a result, transaction ingestion becomes a performance challenge while maintaining fault-tolerance for distributed credit card systems.

As the number of e-payment methods increases, credit card networks must also find new ways to effectively detect fraudulent transactions. While ML tools are useful for noticing patterns and anomalies within a set of data, deploying this to real-time transactions at scale is an immense challenge. Despite the challenges, credit card networks have a responsibility to protect their users. There is also an obvious financial incentive for credit card networks to have adept strategies for fraud detection compared to their competitors.

This paper aims to examine why fault tolerance is essential in credit card networks, propose a distributed system for transaction processing, and provide fraud detection through an ML-based algorithm.

2 Background and Motivation

Credit card networks require robust fault-tolerance to handle the high volume of transactions and protect users from fraudulent charges. Prior work aims to address this with the development of middleware for Byzantine-Fault-tolerance (BFT) for extreme cases [1,4]. Increasingly, there is interest in applying ML techniques for fraud detection to ensure safe transactions for all users within a credit network. However, these ML approaches must deal with the challenges of performance latency and scalability for the high-volume of transaction [2]. Furthermore, replication is an essential technique for providing availability in a fault-tolerant system. In particular, synchronous replication provides no data loss but because of latency, it is limited by distance and bandwidth, which is an issue for geographically distant transactions [3].

To address the described challenges of a credit card network, we propose a distributed system that includes:

I. Passive replication of transaction ingest
II. ML fraud detection algorithm for fraudulent transactions
III. Consensus mechanism for achieving consensus among three independent payment processors.

2.1 Passive Replication

Passive replication is a useful mechanism to provide reliability and consistency in a credit card network. In passive replication, a single primary server handles all client requests, while one or more backup servers receive state updates. This is different from active replication in which all computations are replicated across all servers and the front end must have some set protocol to manage conflicting responses. With passive replication, the failure of the primary server allows for any one of the backup servers to take over and become the new primary. This provides a reliable mechanism for maintaining liveness and availability within the system with minimal loss or delay. This is especially important for credit card networks, which face high volumes of transactions.

Compared to active replication, passive replication is generally less resource-intensive and requires less overhead. Additionally, the system can avoid the synchronization challenges associated with active replication. Ensuring transactions are replicated also helps avoid conflicting or duplicated transactions for merchants.

2.2 ML Fraud Detection

Large language models such as ChatGPT have made transformer-based neural networks one of the most discussed technologies in recent years. Originally proposed in 2017, causal transformer models take a sequence of tokens and compute a prediction for each token, using the knowledge of previous tokens, but not of future tokens [5]. In a language model, this prediction is usually the next word. However, the technology is very general, allowing various kinds of data to be processed by transformer models, such as time series models [6]. For the domain of credit card fraud detection, recent studies show that transformer models have gained traction in the domain of credit card fraud detection, outperforming previous statistical models such as neural networks with simple architectures and tree-based models like XGBoost [7].

In a distributed credit card network, the system must agree on whether to authorize or deny a transaction despite the potential failure of individual components. This agreement is known as consensus. According to the FLP impossibility theorem, reaching exact consensus in a purely asynchronous system with even one faulty process is impossible. To overcome this, our system assumes a model of partial synchrony. We employ a coordinator to manage a quorum consensus. The system requires a majority agreement from independent transaction processors. This ensures valid transactions are correctly authorized. This modular redundancy allows the network to tolerate crash faults similar to established strategies for Byzantine fault-tolerant middleware [1].

3 System Architecture

3.1 Overview

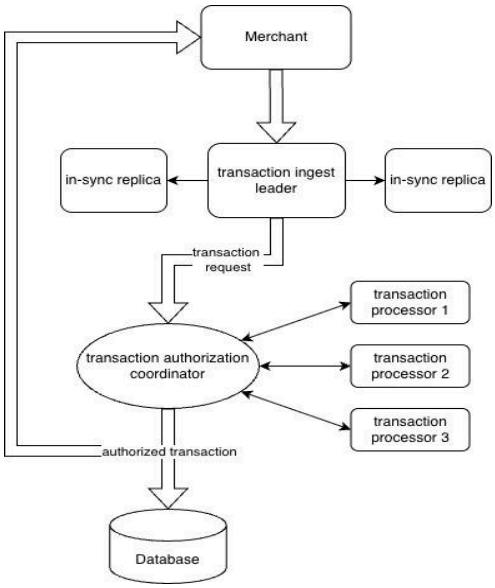

Figure 1: Overview of End-to-End Credit Architecture

The architecture depicted in Figure 1 depicts the end-to-end path of a transaction beginning with the request created by the merchant. The merchant sends incoming transaction requests to the **transaction ingest leader**. The leader forwards transactions to two **in-sync replicas** that replicate each received transaction. If the leader fails, one of these backup replicas can assume its role, ensuring continued ingestion of transactions with minimal delay and loss.

Once a transaction is ingested and replicated, it is forwarded as a **transaction request** to the **transaction authorization coordinator**. This coordinator is responsible for commissioning the three **transaction processors** to independently perform authentication of a given transaction. These processors operate in parallel, each performing computations to determine the validity of the transaction. This includes the ML fraud detection algorithm as well as credit balance checks from the database. Once the coordinator has received results from all three processors, it determines whether the transaction should be approved or denied. A ⅔ majority approval must be attained from the processors in order for the transaction to be authorized by the coordinator. Note that the coordinator uses timeouts in case a processor has failed, providing some synchrony to transaction authorization.

If the transaction is authorized by the coordinator, it is returned to the merchant and published to the **database**, which safely stores all credit balances and transactions. If the transaction is not authorized, the merchant is notified of a rejected transaction. The database ensures that authorized transactions are committed reliably and can be retrieved later for transaction authorization. Overall, the architecture combines replication, coordination, and fraud detection to achieve high availability and strong consistency.

3.2 Causal Transformer Fraud Detection Model

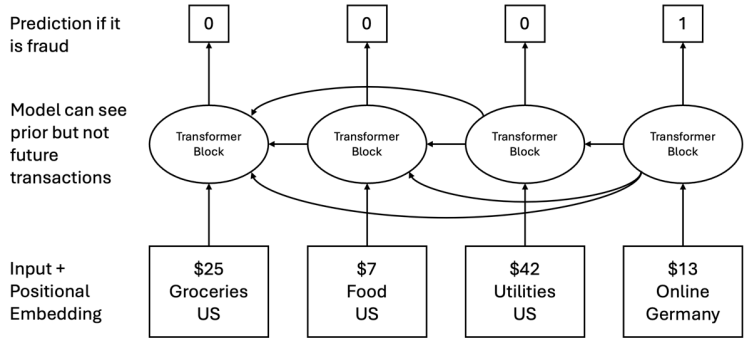

Figure 2: Causal Transformer Model Architecture for Fraudulent Transactions

The question of whether a transaction is fraudulent can only be answered if context is provided. For example, an online shopping transaction in Germany could be absolutely legit if that aligns with the user's general spending habits, but if the user did in-person transactions in the US, and suddenly such a transaction request comes like in Figure 2, fraud is likely.

Therefore, this implementation uses neural networks with a causal transformer architecture to provide a context window to judge the validity of the transaction. In particular, transactions are treated as tokens, containing information on the amount, the category, the country, and the timestamp, given as a difference to the timestamp of the previous transaction. These features get encoded into a 20-dimensional vector. Additionally, positional embedding is added to that vector.

Causal transformers use an attention mask that allows each token to access information from previous tokens but not future tokens for the prediction, modelling the semantic that a decision must be made instantaneously without knowledge of future transactions. At the end of the network, a binary prediction is made for each token, the decision if the transaction shall be classified as fraudulent or legit.

For this model, we select a context size of 128. If there are more transactions, only the last are selected as the context for the requested transaction. Apart from that, the model has 3 attention blocks with one attention head per block. That gives a total parameter count of 20k parameters – a tiny size compared to large language models, but a very effective size for this kind of structured data.

With this solution, the fraud detection is fully ML-based. No explicit rules are given. Instead, all knowledge is derived from the training data. In this project, we fully relied on synthetic data, data that we generated according to the following rules:

- First, a comprehensive set of users is generated. They have a home country, a willingness to travel, an income, and their own preferences on spending in different categories. These attributes influence the spending history of that user.
- For every user, a history gets sampled. During the sampling, a variable is maintained of whether the user is in a different country. Depending on the willingness to travel, this happens more or less frequently. Moreover, at the times of country switches, travel-related purchases are more likely, and online purchases are less likely abroad.
- About 5% of all transactions are fraudulent. In contrast to the legit transactions, they don't follow the user's distribution, but they are completely random in their own distribution. Fraudulent transactions hit all users with the same probability, and they are posted in all countries with the same probability. Fraudulent transactions are more likely to be online transactions.

3.3 Implementation

The principal tool used for transaction communication was Kafka. Three brokers were employed to manage the topics for transaction requests, authorization, and notifications to merchants. Topics and logs were configured with REPLICATION_FACTOR=3. This allowed the transaction processing to continue in case of a failure. Transactions were stored in a postgres database. The entire system could be easily brought up using docker compose. Messages logs and received topics were presented in a UI through port 8080 on the localhost.

3.3.1 Transaction Processing Path

Once transactions were ingested and stored by the transaction ingest leader, they were sent to the transaction authorization coordinator.

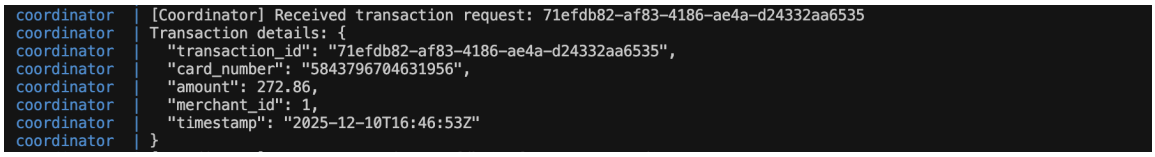

Figure 3: A transaction is received by the transaction authorization coordinator

Figure 3 shows the receipt of a transaction by the coordinator. Each transaction could be uniquely identified by the UUID. Other information regarding the amount, card number, and timestamp of the transaction were included.

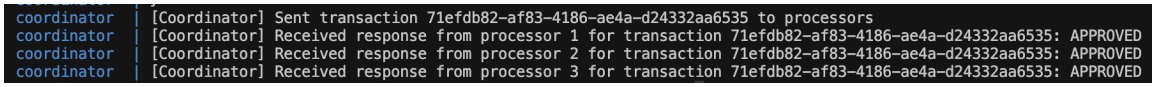

Figure 4: The coordinator sends the transaction to the processors for authentication

The coordinator then sent the received transaction to the three processors for payment authentication (Figure 4). Timeouts were employed for partial synchrony in order to handle failed or slowed processors.

Figure 5: Example of a 3/3 approved transaction

The coordinator mandated ⅔ approval from the processors to authorize a transaction. Figure 5 exemplifies a transaction that was authenticated by all three processors as non-fraudulent and safe in relation to stored credit balances in the database.

In the presence of a failed processor, the coordinator could continue authorizing so long as the remaining two processors approved the transaction. If more than one processor failed at a given time, transactions could no longer be authorized. Processors could deny transactions if the ML algorithm detected fraud. Figure 6 shows a transaction that was

denied by all three processors. In such cases, the merchant was notified of a rejected transaction.

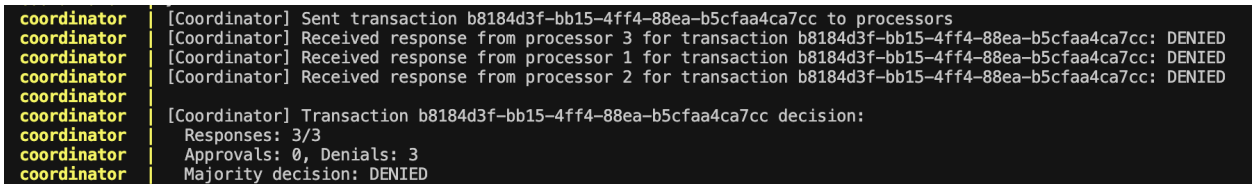

Figure 6: Example of a 3/3 denied fraudulent transaction

4 Performance Evaluation

The model was trained for two hours on a MacBook with data from 80,000 simulated users, and a history of 128 transactions for each of these users. 5% of the transactions are fraudulent, and 95% are legit. For a scenario with such a high class-imbalance, it is typical to discuss the model's performance via conditional probabilities, such as specificity and sensitivity. That is the probability that a fraudulent transaction actually gets classified as fraudulent. Specificity, on the other hand, is the probability that a non-fraudulent transaction actually gets approved. Even though we would like a model to achieve 100% for both scores, there is always a tradeoff between these two, and it is up to the specification to define which property is more important. In the model, it can be controlled via the weight of the two classes in the loss function.

In our modeling, the weight factor is five, meaning that a false-negative is five times as bad as a false positive. That makes the model moderately pessimistic. With this configuration, the model has a sensitivity of 70.2% and a specificity of 96.7%.

5 Future Work

Decentralization of the Coordinator to Avoid Single Points of Failure (SPOF). In the current system, the transaction authorization coordinator is a SPOF. Future work could avoid such failures by decentralizing the coordinator. Although there is fault tolerance within the three processors, relying on a single coordinator is a limitation within the current setup: failure of the coordinator would result in a complete inability to authorize transactions. Future work could explore distributing the coordinator's role across multiple nodes, using decentralization methods to ensure that no single node is responsible for all authorization decisions. With a more decentralized system, incorporating a leader election algorithm like Raft or Paxos would enable the system to respond automatically to node failures, new node additions, and network partitions. This redesign would not only eliminate the coordinator as a SPOF but also enhance system scalability and resilience under heavy loads or partial network failures.

Increased Merchants With Enhanced Communication Channels. The proposed system currently only supports a single merchant. As transaction volumes grow, the system

must support a larger number of merchants concurrently. Future enhancements could extend the messaging infrastructure to handle higher concurrency and improve throughput through load balancing and partitioned message queues. These improvements would provide greater flexibility, enable parallel transaction ingestion, and allow the system to scale to more realistic transaction demands of the global financial network.

ML-Generated Credit Reports for Merchants. Currently, users have to decipher vague transaction descriptions when given their monthly credit reports. Beyond core transactional functionality, the proposed system could also incorporate ML to generate user-friendly credit reports for merchants. Such reports could summarize charges, detect spending trends, warn of possible fraudulent charges, and provide insights about customer behavior. It could also be used to notify the customers of hidden monthly subscriptions they may be unaware of. Integrating this feature would pose a number of privacy and security challenges, but it could enhance the user experience of merchants and their customers.

6 Conclusion

We designed and implemented a distributed credit card network that prioritizes fault tolerance, data consistency, and security. The system successfully mitigates common distributed system challenges by integrating passive replication for transaction ingestion [3] and a coordinator-based consensus mechanism for authorization.

Our architecture demonstrates that high availability can be maintained without sacrificing fraud detection capabilities. The model achieved a specificity of 96.7% and a sensitivity of 70.2% on synthetic data. This confirms that deep learning techniques can be coupled with distributed systems to filter fraudulent activity in real-time. This addresses common challenges of latency and scalability in financial fraud detection [2]. Ultimately, this system highlights the critical trade-offs in distributed financial networks. We must balance the latency required for ML inference against the need for high-throughput transaction processing.

References

[1] M. G. Merideth, Arun Iyengar, T. Mikalsen, S. Tai, I. Rouvellou and P. Narasimhan, "Thema: Byzantine-fault-tolerant middleware for Web-service applications," 24th IEEE Symposium on Reliable Distributed Systems (SRDS'05), Orlando, FL, USA, 2005, pp. 131-140, doi: 10.1109/RELDIS.2005.28.

[2] P. B. Nagaraj, V. R. Vallu, A. Chaluvadi, W. Pulakhandam, P. M. Kumar and A. Antonidoss, "Integrating MLOps, SRE, and Chaos Engineering for Scalable and Resilient Credit Card Fraud Detection Systems," 2025 3rd International Conference on Sustainable Computing and Data Communication Systems (ICSCDS), Erode, India, 2025, pp. 9-14, doi: 10.1109/ICSCDS65426.2025.11167755.

[3] Assaf Natanzon and Eitan Bachmat. 2013. Dynamic Synchronous/Asynchronous Replication. ACM Trans. Storage 9, 3, Article 8 (August 2013), 19 pages. https://doi.org/10.1145/2508011

[4] Neda Soltani Halvaiee, Mohammad Kazem Akbari, A novel model for credit card fraud detection using Artificial Immune Systems, Applied Soft Computing, Volume 24, 2014, Pages 40-49, ISSN 1568-4946, https://doi.org/10.1016/j.asoc.2014.06.042.

[5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser and I. Polosukhin, "Attention Is All You Need," arXiv e-prints, 2023. [Online]. Available: https://arxiv.org/abs/1706.03762

[6] G. Zerveas, S. Jayaraman, D. Patel, A. Bhamidipaty and C. Eickhoff, "A Transformer-based Framework for Multivariate Time Series Representation Learning," arXiv e-prints, 2020. [Online]. Available: https://arxiv.org/abs/2010.02803

[7] C. Yu, Y. Xu, J. Cao, Y. Zhang, Y. Jin and M. Zhu, "Credit Card Fraud Detection Using Advanced Transformer Model," arXiv e-prints, 2024. [Online]. Available: https://arxiv.org/abs/1706.03762